



Universiteit
Leiden
The Netherlands

Data Science & AI

Investigating Local Robustness of TabPFN on Small Numerical Binary Classification Tasks

Willem Scholten

Supervisors:

Dr. Jan van Rijn & Prof. Dr. Holger Hoos

BACHELOR THESIS

Leiden Institute of Advanced Computer Science (LIACS)

www.liacs.leidenuniv.nl

31/03/2026

Abstract

Adversarial robustness has been extensively studied for image classifiers, but remains critically underexplored for tabular models, despite their broad deployment in safety-sensitive domains such as clinical diagnostic support and fraud detection. TabPFN is a recently proposed foundation model for classification problems that leverages in-context learning, obtaining state-of-the-art performance without task-specific training. Existing work on TabPFN’s robustness is limited to single perturbation budgets and non-gradient-based attack methods. This thesis investigates the local adversarial robustness of TabPFN V2.0 on numerical binary classification tasks, comparing it against logistic regression, SVMs, and both standard and adversarially trained MLPs. We evaluate robustness using FGSM and PGD attacks across a range of perturbation budgets on five synthetic datasets and the Wisconsin Breast Cancer dataset, and additionally compute empirical robustness distributions to provide a finer-grained vulnerability assessment. Our central finding is that TabPFN exhibits notably higher robustness than all baselines under single-step FGSM attacks for certain perturbation budgets, but that this advantage largely disappears under iterative PGD on synthetic data, suggesting that TabPFN’s gradient landscape obstructs single-step attacks. On the real-world dataset, however, the robustness advantage persists under both attack types. Robustness distributions further reveal that vulnerability increases with feature dimensionality. These findings deliver the first distribution-level robustness analysis of TabPFN, offering a more complete picture of its adversarial vulnerability than previously available and highlighting the need for further investigation.

Contents

1	Introduction	1
2	Related Work	3
3	Background	4
3.1	Local Robustness	4
3.1.1	Heuristic Attack Methods	4
3.1.2	Robustness Distributions	7
3.2	Automated Classifiers	7
3.2.1	TabPFN	7
3.2.2	Baseline Models	9
4	Experimental Setup	11
5	Computational Scaling Experiments	12
6	Robustness Evaluation Experiments	16
6.1	Synthetic Datasets	16
6.2	Real World Datasets	18
7	Conclusions and Further Research	23
	References	26
A	Detailed Results for Synthetic Robustness Experiments	27
A.1	Classical Methods Comparison	27
A.2	Neural Network Comparison	27

1 Introduction

Performance of automated classifiers has become increasingly important, with applications in autonomous driving [CJB⁺24], medical imaging and diagnostics [LFK⁺19], and face recognition [OC21]. Deep neural networks have achieved human-level accuracy in such classification tasks. However, minimal perturbations to an input can lead to misclassifications by the network. These perturbations, called adversarial attacks, are vulnerable to exploitation [SZS⁺13]. For example, in autonomous driving systems, minor perturbations to a stop sign can lead to misclassifications, potentially resulting in dangerous traffic violations and accidents.

Existing research on adversarial attack vulnerability has predominantly focused on image classifiers, where several methods have been developed to construct adversarial examples heuristically. This thesis employs two such methods: the Fast Gradient Sign Method (FGSM) [GSS14], a single-step gradient-based approach, and Projected Gradient Descent (PGD) [MMS⁺17], an iterative extension of FGSM. Models that achieve strong performance on standard benchmarks have been shown to be susceptible to such attacks, as illustrated in Figure 1. Alongside heuristic methods, formal verification techniques have been proposed to characterize a model’s vulnerability to adversarial perturbations with guarantees (e.g., [KBD⁺17, TXT17]). However, their computational cost renders them infeasible for this work, motivating the use of heuristic approaches.

While research has focused on image classification, robustness guarantees are equally critical for automated classifiers operating on other data types. In particular, tabular classifiers remain under-explored from a robustness perspective. Consider, for example, clinical decision-support systems based on electronic health records, where minor perturbations in laboratory values or patient attributes could lead to inappropriate treatment decisions due to incorrect classifications.

Motivated by the disparity between the practical importance of adversarial robustness for tabular models and the limited attention it has received in the literature, we investigate the local robustness properties of TabPFN, a recently proposed transformer-based classifier designed for tabular data [HMEH23]. Unlike conventional supervised learning approaches that train a separate model for each dataset, TabPFN is pre-trained once on a large collection of synthetically generated tabular datasets. The model uses in-context learning, in which it receives both labeled training and unlabeled test samples as input to perform training and inference simultaneously.

Before exploring the robustness of TabPFN via FGSM and PGD attacks, this work first investigates the computational cost of these attacks on TabPFN and other machine classification models. Having established the computational limits of our experimental setup, we proceed to perform a comparative robustness analysis of TabPFN against SVMs, logistic regression, and both adversarially and non-adversarially trained MLPs on synthetic data. Then, to probe the effect of feature dimensionality on robustness, we find robustness distributions of TabPFN across different feature dimensions. To conclude our analysis, we turn to a real-world dataset and compare TabPFN against SVMs and logistic regression, then inspect the robustness distribution of TabPFN in this more realistic setting.

Organization Section 2 outlines relevant research to this thesis. Section 3 introduces all necessary concepts, starting by formally defining local robustness, followed by a formal introduction of FGSM

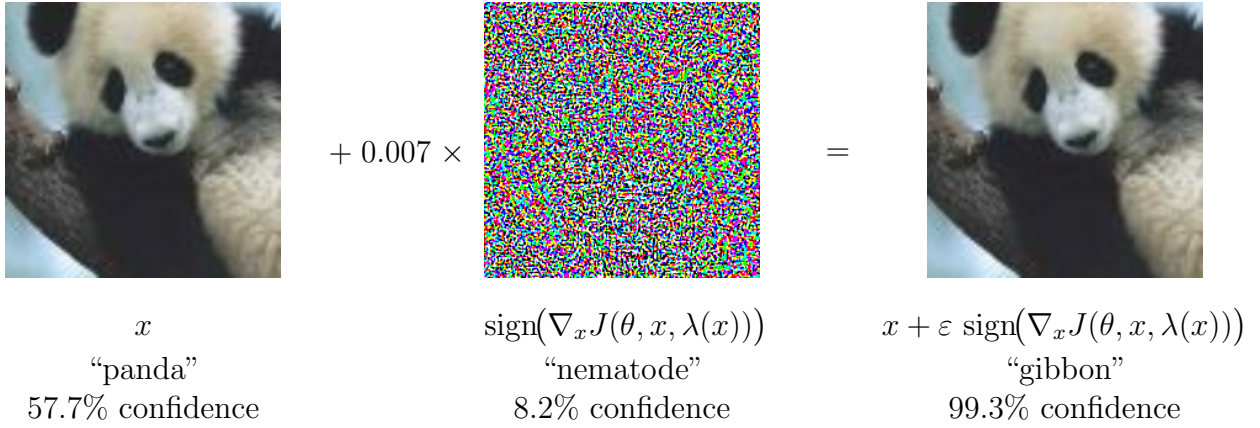


Figure 1: Example of an FGSM perturbation: original input, signed gradient perturbation, and resulting adversarial example, obtained from [GSS14].

and PGD, robustness distributions, and all automated classifiers used in this work. Section 4 details the experimental setup. Section 5 follows by reporting on two experiments regarding computational scaling. The results of the robustness experiments are written in Section 6. Finally, Section 7 concludes with a concise summary of the results and suggests directions for further research.

Main Results Our experiments show that TabPFN consistently exhibits lower vulnerability than logistic regression, SVMs, and MLPs under single-step FGSM attacks on synthetic data, but that this advantage disappears under the stronger iterative PGD attack. On the Wisconsin Breast Cancer dataset, however, TabPFN retains a substantial robustness advantage under both attack types. Robustness distributions further reveal that vulnerability increases with feature dimensionality, suggesting that TabPFN’s robustness advantage on the real-world dataset is not simply a consequence of its dimensionality but rather of the data’s structure.

Code For reprod

2 Related Work

Because TabPFN has only recently been proposed, research on its robustness remains limited. Work on the subject focuses on tree-based ensembles, leaving gradient-based machine learning methods unexplored. Below, we present the most current research.

Djilani et al. [DST+25] present the first comprehensive evaluation of the adversarial robustness of tabular foundation models, evaluating TabPFNV2 and TabICL. They use the TabularBench benchmark with different attack mechanisms on datasets from finance, cybersecurity, and healthcare to demonstrate that both foundation models are more vulnerable to evasion attacks than the best-performing specialized models. The authors also show that tabular foundation models can serve as surrogates for generating transferable adversarial examples against conventional models such as XGBoost and random forests.

To mitigate these vulnerabilities, the authors propose Adversarial In-Context Learning (AICL), a training method that exploits tabular foundation models’ in-context learning by iteratively replacing context samples with adversarially perturbed instances, while leaving the model weights unchanged. They show that AICL consistently outperforms standard adversarial fine-tuning for tabular foundation models, improving robust accuracy on LCLD from 12.5% to 37.2% for TabICL. However, adversarially trained foundation models do not achieve the robustness of similarly trained specialized models.

Although this work is closely related to ours, it differs in several points. We constrain our analysis to binary classification of numerical data and consider gradient-based attack methods rather than tree-based ones. Most importantly, where their work evaluates robustness at a single perturbation budget, we extend the analysis to a range of budgets and additionally compute empirical robustness distributions across multiple datasets. This provides a more complete picture of how TabPFN’s adversarial vulnerability varies with perturbation strength, rather than a single snapshot at one fixed budget.

3 Background

We now formally introduce all concepts required for this thesis. We begin with a mathematical definition of adversarial robustness, followed by the narrower formulation we adopt throughout this work. We then describe the FGSM and PGD attack methods, accompanied by pseudocode and a geometric illustration. Finally, we introduce robustness distributions and the classifiers considered in our experiments.

3.1 Local Robustness

We first present a formal definition of adversarial robustness, following the notation in [TXT17] closely. Consider a deep neural network classifier $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $n, m \in \mathbb{Z}_{\geq 1}$, that takes an input $x \in \mathbb{R}^n$, and outputs a score vector $s = N(x) \in \mathbb{R}^m$, where s_k is the score for class label $k \in \{1, \dots, m\}$. Let $\mathcal{A}(x)$ denote the set of all allowable perturbations for an input x , let $\lambda(x)$ denote the true class label of input x , and let $\mathcal{X}_{\text{valid}}$ be the domain of valid inputs for N . The network N is said to be locally robust to perturbations on x_0 iff the following holds:

$$\forall x \in (\mathcal{A}(x_0) \cap \mathcal{X}_{\text{valid}}) : \arg \max_i f_i(x) = \lambda(x_0). \quad (1)$$

This definition is general in that it leaves open the choice of $\mathcal{A}(x)$ and $\mathcal{X}_{\text{valid}}$. Throughout this work, we take the domain of valid inputs $\mathcal{X}_{\text{valid}}$ to be all of $\mathbb{R}^n$, though we discuss alternatives in Section 7. For the allowed perturbation set, we consider ℓ_∞ balls of varying radius ε around x_0 , so that (1) reduces to

$$\forall x \in \{x : \|x - x_0\|_\infty \leq \varepsilon\} : \arg \max_i f_i(x) = \lambda(x_0). \quad (2)$$

3.1.1 Heuristic Attack Methods

Various methods have been developed to assess local robustness in automated classifiers. Adversarial attacks constitute a relatively straightforward approach for assessing the robustness of gradient-based classifiers, whereby a small perturbation is applied to an input with the objective of inducing a misclassification [SZS⁺13]. Such attacks leverage the gradient of the loss function with respect to the input to identify directions in the input space along which small perturbations yield the largest increase in classification error.

FGSM attack Possibly the simplest of such methods is the Fast Gradient Sign Method (FGSM) [GSS14]. Given a model with parameters θ , input x , target classification $\lambda(x)$, and cost function $J(\theta, x, \lambda(x))$, this process produces an adversarial example x_{adv} by calculating

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, \lambda(x))), \quad (3)$$

where $\nabla_x J(\theta, x, \lambda(x))$ is the gradient of the loss with respect to the input x , and $\varepsilon > 0$ controls the magnitude of the perturbation. The sign operator is applied element-wise, yielding a perturbation that maximally increases the loss under an ℓ_∞ -norm constraint. The example in Figure 1 was created using FGSM, while the left panel in Figure 2 shows the process geometrically. The procedure

is also described in Algorithm 1.

In the context of automated image classification, this method reliably produces adversarial examples that induce misclassification, as illustrated in Figure 1. The authors of [GSS14] demonstrate that, for $\varepsilon = 0.25$, this procedure causes a shallow softmax classifier to exhibit an error rate of 99.9% on the MNIST test set, with an average confidence of 79.3% assigned to the incorrect labels. Under the same perturbation magnitude, a maxout network misclassifies 89.4% of the adversarial examples, with an average confidence of 97.6%. Similarly, for $\varepsilon = 0.1$, they report an error rate of 87.15% and an average confidence of 96.6% on adversarial examples generated for a convolutional maxout network evaluated on a preprocessed version of the CIFAR-10 dataset.

Algorithm 1: Fast Gradient Sign Method (FGSM)

Input: Model parameters θ , input x , true label $\lambda(x)$, perturbation magnitude $\varepsilon > 0$, loss function J

Output: Adversarial example x_{adv}

$g \leftarrow \nabla_x J(\theta, x, \lambda(x));$

$x_{\text{adv}} \leftarrow x + \varepsilon \cdot \text{sign}(g);$

return x_{adv}

PGD attack A second procedure that builds on the same concepts as FGSM is the Projected Gradient Descent (PGD) attack [MMS⁺17]. PGD is an iterative extension of FGSM, and refines an adversarial input by repeatedly ascending the loss landscape while projecting the proposed perturbation onto the allowed region.

Given a network with parameters θ , an input $x \in \mathcal{X}_{\text{valid}}$ with true label $\lambda(x)$, and a perturbation budget $\varepsilon > 0$, the PGD attack seeks an adversarial example via iterative projected gradient ascent. Starting from an initial point $x^0 \in \mathcal{A}(x)$, the update rule at iteration t is given by

$$x^{t+1} = \Pi_{\mathcal{A}(x) \cap \mathcal{X}_{\text{valid}}} \left(x^t + \alpha \cdot \text{sign} \left(\nabla_x J \left(\theta, x^t, \lambda(x) \right) \right) \right), \quad (4)$$

where $\alpha > 0$ is the step size, and $\Pi_{\mathcal{S}}(\cdot)$ denotes the projection operator onto the set $\mathcal{S}$. The projection ensures that each iterate remains both within the allowable perturbation region and the domain of valid inputs. After a fixed number of iterations T , the final iterate x^T is returned as an adversarial example. The procedure is geometrically represented in the right panel of Figure 2. To reduce sensitivity to the starting point, we repeat this iterative procedure over R random restarts, where each initial point x^0 is drawn uniformly at random from the allowable perturbation region $\mathcal{A}(x)$, and retain the adversarial example that achieves the highest loss. This complete process is described in Algorithm 2.

While adversarial attacks are effective at demonstrating the existence of robustness violations, they do not provide formal guarantees regarding the absence of adversarial examples and therefore motivate the development of more rigorous verification-based approaches. Such techniques have been proposed for various linear [TXT17] and transformer-based architectures [BDBV21], but they

Algorithm 2: Projected Gradient Descent (PGD) Attack with Random Restarts

Input: Model parameters θ , input $x \in \mathcal{X}_{\text{valid}}$, true label $\lambda(x)$, perturbation budget $\varepsilon > 0$, step size $\alpha > 0$, number of iterations T , number of restarts R , loss function J

Output: Adversarial example x^*

$x^* \leftarrow x$;

for $r = 1$ **to** R **do**

$\delta \sim \mathcal{U}(-\varepsilon, \varepsilon)^d$;

$x^0 \leftarrow \Pi_{\mathcal{A}(x) \cap \mathcal{X}_{\text{valid}}}(x + \delta)$;

for $t = 0$ **to** $T - 1$ **do**

$g \leftarrow \nabla_x J(\theta, x^t, \lambda(x))$;

$\tilde{x}^{t+1} \leftarrow x^t + \alpha \cdot \text{sign}(g)$;

$x^{t+1} \leftarrow \Pi_{\mathcal{A}(x) \cap \mathcal{X}_{\text{valid}}}(\tilde{x}^{t+1})$;

if $J(\theta, x^T, \lambda(x)) > J(\theta, x^*, \lambda(x))$ **then**

$x^* \leftarrow x^T$;

return x^*

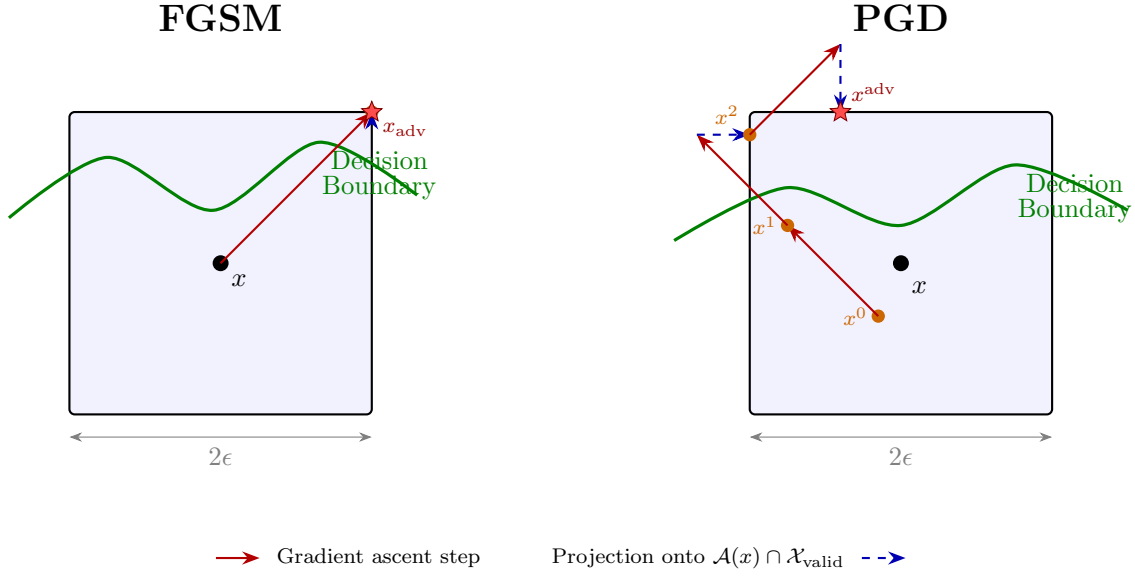


Figure 2: Geometric illustration of the FGSM and PGD methods in a two-dimensional input space. The blue region denotes $\mathcal{A}(x)$, which we assume fully lies in $\mathcal{X}_{\text{valid}}$. The green lines represent the decision boundary.

remain computationally expensive, rendering them infeasible for large-scale models. Consequently, in this work, we focus on the aforementioned heuristic attack methods.

3.1.2 Robustness Distributions

Robustness distributions provide a way to visualize a classifier’s adversarial robustness across an entire test set. We closely follow the definition and procedure introduced in [BBHVR25]. For a given sample x_0 , we define its robustness as the minimum perturbation budget ε^* for which an attack succeeds in inducing a misclassification. For samples that are misclassified on the clean input, we set $\varepsilon^* = 0$, since no perturbation is required to induce a misclassification. The robustness distribution is then obtained by computing ε^* for every sample in the test set and plotting the empirical cumulative distribution function. A distribution that rises steeply at small ε indicates a highly vulnerable model, while one that remains low for large ε signals greater robustness.

In this work, we approximate ε^* by performing a binary search over a predefined search space. At each step, we run an adversarial attack at the current budget and narrow the interval accordingly, repeating until the search interval falls below a specified stopping margin δ , yielding an approximate $\tilde{\varepsilon} \approx \varepsilon^*$. Because we use heuristic attack methods rather than complete verifiers, it is not guaranteed that $\tilde{\varepsilon} - \varepsilon^* \leq \delta$, since the attack may fail to find an adversarial example even when one exists within the current budget. Consequently, $\tilde{\varepsilon}$ provides an upper bound on the true robustness ε^* , and the approximation error may exceed δ . **QUESTION: can I cite the paper that is still in review?**

3.2 Automated Classifiers

This thesis considers several automated classification methods, which we formally introduce below. We begin with TabPFN, describing its mathematical foundations, training procedure, and architecture. We then provide concise introductions to logistic regression, SVMs, and neural network classifiers.

3.2.1 TabPFN

TabPFN (Tabular Prior-Data Fitted Network) is a transformer-based classifier designed specifically for learning from tabular data. Unlike conventional supervised learning approaches that train a separate model for each dataset, TabPFN is pre-trained once on a large collection of synthetically generated tabular datasets. The model leverages in-context learning, in which it receives both labeled training and unlabeled test samples as input and jointly performs training and inference.

Bayesian Inference The mathematical foundation for TabPFN lies in Bayesian modeling. In this framework, we assume there is a set of data-generating processes Φ that possibly generate our data $\mathcal{D}$, where the prior $p(\phi)$, $\phi \in \Phi$ encodes the researcher’s belief over which process gave rise to the observed data $\mathcal{D}$. Under this paradigm, prediction for a new data point x is obtained by marginalizing over all possible processes, yielding the posterior predictive distribution (PPD)

$$p(y \mid x, \mathcal{D}) \propto \int_{\Phi} p(y \mid x, \phi) p(\mathcal{D} \mid \phi) p(\phi) d\phi, \quad (5)$$

where the terms $p(\mathcal{D} \mid \phi)$ and $p(\phi)$ together form the posterior over ϕ via Bayes' rule. In most cases, it is impossible to find $p(y \mid x, \mathcal{D})$ in closed form. Classical approaches to approximate it, such as Markov Chain Monte Carlo or Variational Inference, are computationally expensive, particularly when repeated for each new dataset. This cost motivates the development of methods that can approximate the posterior predictive distribution more efficiently.

PFNs TabPFN is a Prior-Data Fitted Network (PFN), a class of neural architectures specifically designed for predicting the PPD efficiently. A PFN is trained once on a large collection of synthetic datasets sampled from a prior $p(\phi)$. During this exercise, a data-generating process ϕ is sampled from the prior, and a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ is then drawn from $p(\mathcal{D} \mid \phi)$. The network receives a subset of the dataset as context and is trained to predict the labels of the held-out query points, effectively learning to map any context-query pair $(\mathcal{D}_{\text{train}}, x)$ directly to an approximation of $p(y \mid x, \mathcal{D}_{\text{train}})$. The training loss is cross-entropy on the held-out set.

SCMs Two distinct data-generating paradigms were used for training the first version of TabPFN: Structural Causal Models (SCM) and Bayesian Neural Networks (BNN). TabPFN V2, however, relies exclusively on SCMs for data generation. An SCM consists of a directed acyclic graph (DAG) $\mathcal{G}$ over a collection of structural assignments $Z = \{z_1, \dots, z_k\}$, where each variable is defined by a mechanism

$$z_i = f_i(z_{PA_{\mathcal{G}}(i)}, \epsilon_i), \quad (6)$$

where $PA_{\mathcal{G}}(i)$ denotes the parents of node i in $\mathcal{G}$, f_i is a deterministic function and ϵ_i is a noise variable. To generate a synthetic dataset from this prior, a random SCM is first sampled, including its DAG structure, activation functions, edge weights, and noise distributions. A subset of nodes Z_X is assigned as observed features and a single node z_y as the target. For each sample in the dataset, noise variables are drawn and propagated through the graph.

As an example, consider the three SCMs from a hypothesis space Φ , illustrated in Figure 3. All three SCMs produce datasets $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ in which x and y are correlated, yet the underlying generating process, and therefore the optimal prediction rule, differs in each case. In the Bayesian framework underlying PFNs, ϕ_1 , ϕ_2 , and ϕ_3 are all elements of Φ , each weighted by its prior probability $p(\phi)$. Given a training dataset $\mathcal{D}$, the posterior predictive distribution implicitly assigns higher weight to whichever SCM, or family of SCMs, best explains the observed data.

Architecture The architecture of TabPFN, illustrated in Figure 4, is based on a modified transformer encoder. The input to the model consists of a tabular context: N labeled training samples $(\mathbf{x}_i, y_i)_{i=1}^N$ each with d features and a binary label, together with a query point $\mathbf{x}^*$ whose label is to be predicted. Each sample is represented as a row of $d + 1$ values: the d features plus the label. A feature tokenizer maps each of these $d + 1$ input columns independently into a k -dimensional embedding, producing one token per column per sample. This yields a set of $N + 1$ token sequences that are fed into a stack of self-attention layers. The self-attention mechanism operates jointly over all $N + 1$ samples, allowing the model to learn dependencies between the training context and the query point within a single forward pass. The final representation corresponding to the query

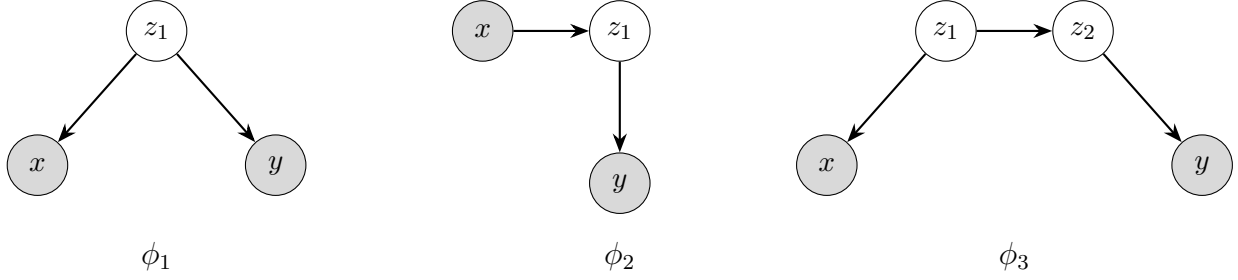


Figure 3: Three example SCMs. Shaded nodes denote observed variables, while white nodes represent latent variables. ϕ_1 : a common latent cause generates both x and y . ϕ_2 : x influences y through a latent mediator z_1 . ϕ_3 : a latent chain connects x and y indirectly through two hidden variables. Figure inspired by Figure 2 in [HMEH23].

point $\mathbf{x}^*$ is passed through a multilayer perceptron classification head, which outputs a predictive distribution over the binary class labels.

3.2.2 Baseline Models

We compare TabPFN against several classical classifiers, which we briefly introduce below.

Logistic regression To establish the robustness of TabPFN, we compare the model against various other methods. A baseline model for binary classification is logistic regression. Logistic regression is a linear probabilistic classification model that estimates the conditional probability of a binary outcome given an input feature vector. Formally, given an input $x \in \mathbb{R}^d$, logistic regression models the probability of class membership as

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \quad (7)$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ denote the model parameters, and $\sigma(\cdot)$ is the logistic sigmoid function. The predicted class label is obtained by thresholding this probability, typically at 0.5. Model parameters are learned by minimizing the negative log-likelihood of the observed labels, which is equivalent to minimizing the binary cross-entropy loss over the training data. As a linear classifier, logistic regression induces a linear decision boundary in the input space, making it computationally efficient and well understood, while also serving as a strong baseline for evaluating the robustness properties of more complex models.

Support Vector Classifier A different model to compare against is the support vector classifier (SVC), a margin-based discriminative model that seeks to separate data points of different classes by constructing a decision boundary with maximum geometric margin. In its linear form, given training data $\{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$, the SVC determines a hyperplane parameterized by $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ by solving a convex optimization problem that maximizes the margin between classes while penalizing misclassifications via a regularization parameter. The resulting classifier assigns labels based on the sign $w^\top x + b$.

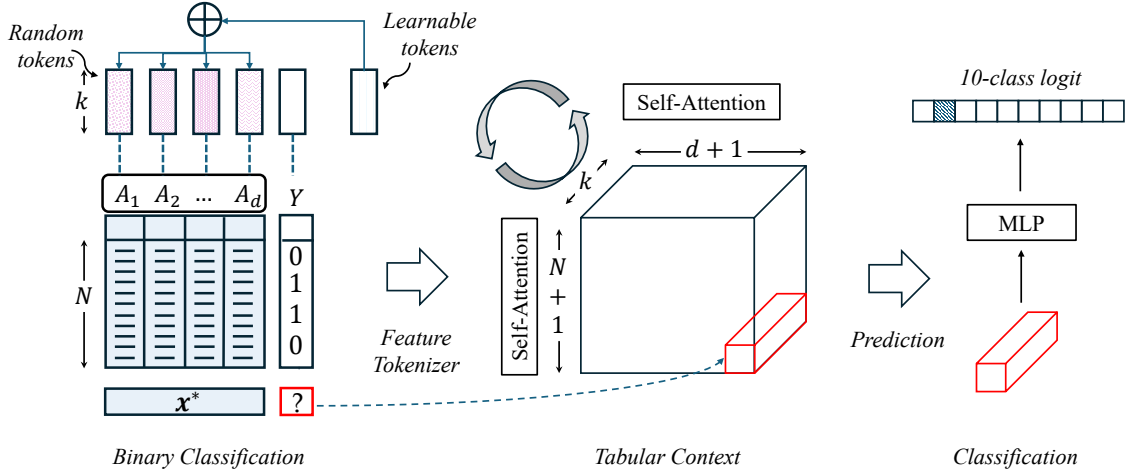


Figure 4: Overview of the TabPFN architecture. Input tabular data consisting of N training samples with d features and labels Y forms the tabular context. Each sample, along with a query point $\mathbf{x}^*$, is passed through a feature tokenizer that maps the $d + 1$ input columns into k -dimensional token embeddings. These tokens are then processed by a stack of self-attention layers, allowing the model to attend over both training samples and the query. The final representation of the query point is passed through an MLP to produce a classification prediction. This figure is obtained from [YLC].

Neural Network Classifier A simple neural network classifier extends linear models by introducing one or more nonlinear transformations of the input through the use of hidden layers. We employ a two-layer neural network comprising an input layer, a single hidden layer with nonlinear activation functions, and an output layer that produces class scores or probabilities. Given an input $x \in \mathbb{R}^d$, the network computes a sequence of affine transformations followed by element-wise nonlinearities, enabling it to learn nonlinear decision boundaries in the input space. Model parameters are optimized by minimizing cross-entropy, using gradient-based optimization methods.

Despite its simplicity, this class of models can capture interactions among features that are inaccessible to purely linear classifiers, making it a natural intermediate baseline between classical methods and more complex deep learning architectures like TabPFN. Furthermore, such models can be hardened through adversarial training, providing a meaningful baseline for assessing whether TabPFN exhibits comparable resilience without explicit robustness interventions.

4 Experimental Setup

In this section, we describe the hardware and software specifications used throughout our experiments. We also specify the standard model configurations and formally define the metrics used to quantify robustness.

Attack Model Across all experiments, we perform untargeted attacks under an ℓ_∞ perturbation constraint. For gradient-based methods, the adversary maximizes the binary cross-entropy loss with respect to the true label. For the SVM baseline, which lacks a differentiable loss suitable for gradient-based perturbation, the adversary instead maximizes the distance to the decision boundary. Unless stated otherwise, the FGSM attack uses the specified ε value directly, while the PGD attack is configured with $T = 5$ iterations and 5 random restarts, where we choose $\alpha = 2\varepsilon/T$.

Metrics We use several metrics to quantify robustness. The first is Attack Success Rate (ASR), defined by

$$\frac{\sum_{x_0 \in \mathcal{D}} \mathbb{1}(\arg \max_i f_i(x_{adv}) \neq \lambda(x_0)) \cdot \mathbb{1}(\arg \max_i f_i(x_0) = \lambda(x_0))}{\sum_{x_0 \in \mathcal{D}} \mathbb{1}(\arg \max_i f_i(x_0) = \lambda(x_0))}, \quad (8)$$

where all notation is as previously defined. In words, the ASR measures the fraction of correctly classified samples for which the adversarial attack succeeds in inducing a misclassification. Second, we use adversarial accuracy, defined as the fraction of samples that are both correctly classified and robust to the attack:

$$\frac{\sum_{x_0 \in \mathcal{D}} \mathbb{1}(\arg \max_i f_i(x_{adv}) = \lambda(x_0))}{|\mathcal{D}|}. \quad (9)$$

Experimental Environment All experiments are conducted on the shared vibranium.liacs.nl GPU server equipped with an Intel Xeon Silver 4214R CPU (48 cores, 2.40 GHz), 251 GB of RAM, and two NVIDIA GeForce RTX 3090 GPUs (24 GB VRAM each), running Linux 5.14.0 with CUDA 13.1 (driver version 590.48.01) and Python 3.7.13. Models were implemented using PyTorch 2.10.0, scikit-learn 1.7.2, TabPFN 6.3.2, and NumPy 2.4.2. Unless stated otherwise, TabPFN V2 was used in all experiments. We chose this version over more recent releases for two main reasons. First, it is the most widely validated version: the introductory paper has accumulated over 400 citations, and the repository has been downloaded more than 2,000,000 times. This broad adoption, combined with its more permissive license, makes it the most suitable choice for reproducible and accessible research. Second, this thesis is intended as a preliminary guide for future work. Given the rapid pace of development, TabPFN V2.6 was released as recently as March 24, 2026; we leave the evaluation of newer versions to future work.

5 Computational Scaling Experiments

Before turning to the main objective of this thesis, we first investigate the computational limits of our setup. PGD attacks are notoriously expensive on image classifiers [MMS⁺17], and it is therefore important to establish which experiments are feasible within practical time constraints. To this end, we detail two scaling experiments. The first examines how computation time grows with feature dimension, the second how it scales with dataset size. These experiments serve to establish the computational limits to guide the architecture of the other experiments.

Feature dimensionality scaling As a preliminary experiment, we assess the computational feasibility of our attack methods given the available resources. Specifically, we measure how the runtime of FGSM and PGD attacks scales with the dimensionality of the feature space. For each attack method, we report the per-sample attack time, total runtime, and the ratio of FGSM to PGD execution time, alongside the corresponding TabPFN training time across all feature dimensions. We conduct this analysis across multiple runs on a single dataset and report mean runtimes with error bars to indicate variability.

We evaluate performance across feature dimensionalities of 2, 10, 20, 30, 40, and 50 on a synthetic dataset comprising 60 samples. For the FGSM attack, we set $\varepsilon = 0.5$. For the PGD attack, we use a step size of $\alpha = 2\varepsilon/T$, with $T = 5$ iterations and 5 random restarts. All experiments are repeated 3 times, and the results are averaged to reduce variance. On the described hardware configuration, the full experiment completes in 134 minutes and 47 seconds. The results are shown in Figure 5.

Panel (a) reveals that per-sample attack time is approximately constant across all feature dimensionalities for both FGSM and PGD, indicating that the computational cost of attacking TabPFN is dominated by the model’s forward and backward pass rather than the dimensionality of the input. Panel (b) confirms this trend for total runtime, which remains stable across feature counts. Panel (c) shows that the empirical PGD/FGSM runtime ratio hovers around 15, well below the theoretical ratio of $\text{steps} \times \text{restarts} = 25$. We believe this discrepancy suggests that each standalone FGSM call incurs fixed overhead, e.g., model initialization or memory allocation, that is reduced when gradient steps are performed iteratively within PGD. Finally, panel (d) shows that TabPFN’s training time stabilizes around 200 ms for 10 or more features, with a brief peak at 2 features likely attributable to initialization costs. Overall, these results suggest that feature dimensionality, at least within the range tested, is not a limiting factor for the computational feasibility of gradient-based adversarial attacks on TabPFN.

Dataset size scaling Second, we perform an analogous experiment to investigate how computational demand scales with dataset size. We report the same metrics as above, now varying the numbers of training and test samples while holding the feature dimensionality fixed, again averaging across three runs for each dataset and displaying error bars.

We evaluate the effect of dataset size on attack runtime by varying the number of samples across 24, 48, 72, and 96, with a fixed feature dimensionality of 5. The FGSM attack uses $\varepsilon = 0.5$, while the PGD attack is configured with a step size of $\alpha = 2\varepsilon/T$, with $T = 5$ iterations, and 5 random restarts. All experiments are again repeated 3 times and the results are averaged to reduce variance.

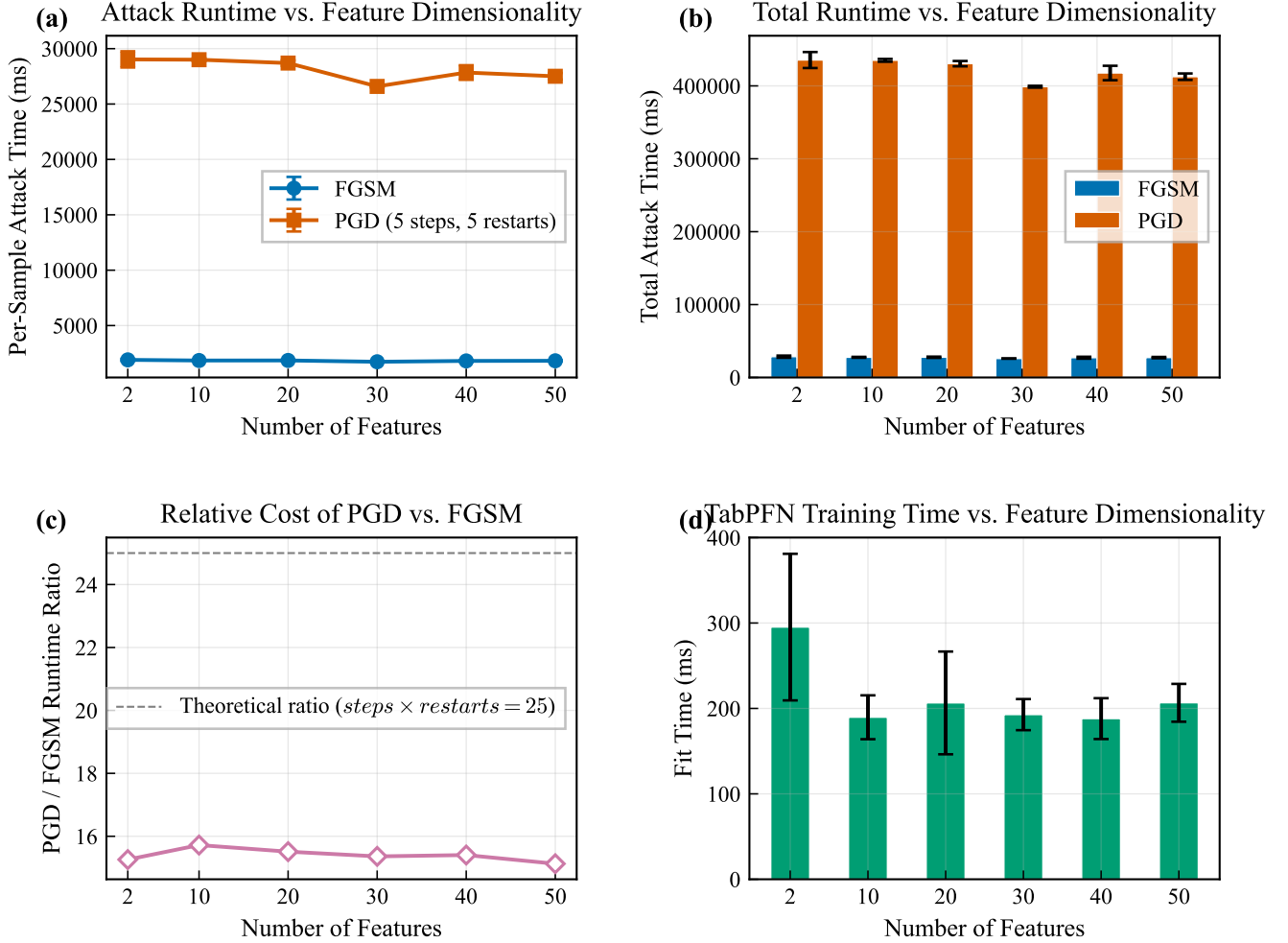


Figure 5: Runtime analysis of adversarial attacks on TabPFN across varying feature dimensionalities. (a) Per-sample attack time for FGSM and PGD (5 steps, 5 restarts), showing that both methods exhibit approximately constant cost with respect to the number of features. (b) Total attack time over the full test set, reflecting the same trend at scale. (c) Observed PGD/FGSM runtime ratio compared to the theoretical ratio of $\text{steps} \times \text{restarts} = 25$. The empirical ratio remains consistently below this bound at approximately 15, suggesting non-negligible fixed overhead per FGSM call, which is reduced across PGD iterations. (d) TabPFN training (fit) time as a function of feature dimensionality, which stabilizes around 200 ms after an initial peak at 2 features. Error bars denote standard deviation over 3 repetitions.

On the described hardware configuration, the full experiment completes in 109 minutes and 27 seconds. The results are shown in Figure 6.

Panel (a) shows that the FGSM per-sample attack time remains approximately constant at 2 seconds across datasets, whereas the PGD per-sample time increases moderately from 30 to 36 seconds. We believe this increase is attributable to the TabPFN design, which conditions on the entire training set at each forward pass. This means that larger training sets result in more expensive gradient computations per sample. Panel (b) reflects both this increase and the increase in test samples to be attacked, both of which inflate the total runtime. Panel (c) shows that the PGD/FGSM runtime ratio remains below the theoretical bound of 25, though it exhibits a notable spike at 72 samples before dropping again at 96. **TODO: REASON** Panel (d) reveals that TabPFN training time remains stable around 200–350 ms for datasets up to 72 samples but increases sharply to approximately 1250 ms at 96 samples, with considerably higher variance. **TODO: REASON**. Taken together, these results indicate that dataset size, unlike feature dimensionality, has a meaningful impact on both attack cost and model fitting time, and should be considered when designing larger-scale experiments.

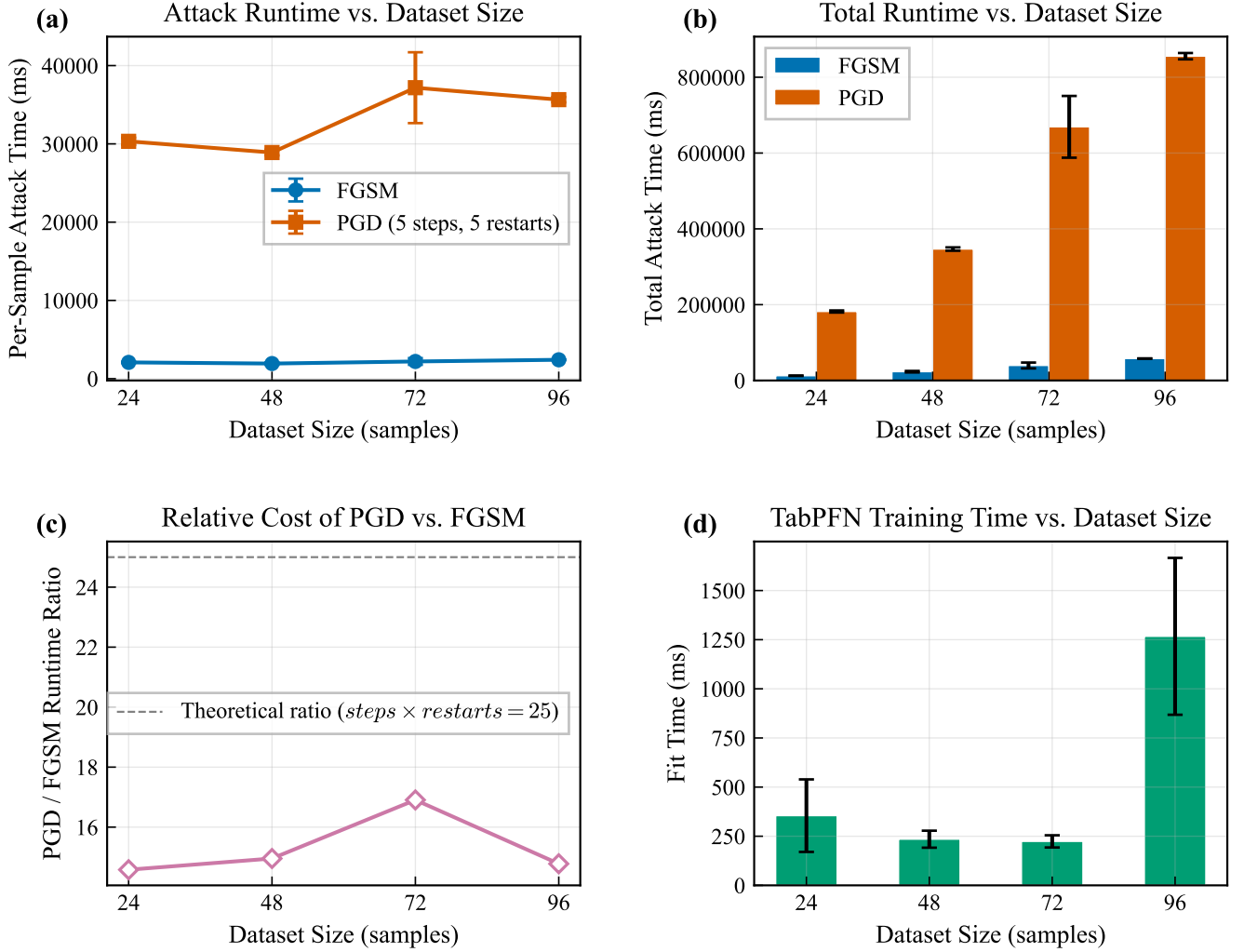


Figure 6: Runtime analysis of adversarial attacks on TabPFN across varying dataset sizes with a fixed feature dimensionality of 5. (a) Per-sample attack time for FGSM and PGD (5 steps, 5 restarts), showing that FGSM remains approximately constant while PGD exhibits a moderate increase for larger datasets. (b) Total attack time over the full test set, which grows with dataset size due to the increasing number of test samples. (c) Observed PGD/FGSM runtime ratio compared to the theoretical ratio of $\text{steps} \times \text{restarts} = 25$. The empirical ratio remains below this bound, with a notable spike at 72 samples. (d) TabPFN training time as a function of dataset size, which remains stable for smaller datasets but increases sharply at 96 samples, suggesting a transition in the model’s internal computation. Error bars denote standard deviation over 3 repetitions.

6 Robustness Evaluation Experiments

Having established the computational limits of our setup, we proceed to the central aim of this thesis. We first describe a comparative robustness analysis of TabPFN against SVMs, logistic regression, and both adversarially and non-adversarially trained MLPs on synthetic data, followed by an investigation of the robustness distribution of TabPFN across different feature dimensions. We then turn to a real-world dataset, where we again compare TabPFN against SVMs and logistic regression, after which we inspect the robustness distribution of TabPFN in this more realistic setting.

6.1 Synthetic Datasets

In this section, we specify all experiments conducted on the synthetic datasets. All datasets are generated with scikit-learn’s `make_classification` method: `make_classification`. For each dataset, all features are informative (`n_informative = n_features`), with no redundant or repeated features (`n_redundant = 0`, `n_repeated = 0`), one cluster per class, and class separation `class_sep = 2.0`. Features are standardized to zero mean and unit variance using `StandardScaler`, fit on the training split only. Train-test splits are stratified by class label using a 75/25 split. Unless stated otherwise, we use 5 features.

Comparison With Classical Methods We now compare adversarial robustness across TabPFN, Logistic Regression, and a linear SVM. For different values of ε , we investigate the fraction of test samples where a misclassification is induced under both FGSM and PGD attacks for each model. This allows us to assess whether TabPFN is more or less vulnerable to adversarial perturbations than conventional classifiers. We report mean attack success rates across datasets for all five models, accompanied by error bars. Per-dataset breakdowns are provided in Appendix A.1. On the described hardware configuration, the full experiment completes in approximately 15 hours and 58 minutes. The results are shown in Figure 12.

At small perturbation budgets ($\varepsilon \leq 0.5$), all three models exhibit comparably low attack success rates under both FGSM and PGD, suggesting that minor input perturbations are insufficient to cross any of the learned decision boundaries. As ε increases beyond this threshold, however, a clear divergence emerges under FGSM: logistic regression and the linear SVM become substantially more vulnerable, while TabPFN maintains a notably lower ASR, indicating greater robustness to single-step gradient attacks. Under PGD, this robustness gap disappears, with all three models reaching similar ASR levels at large ε . This suggests that TabPFN’s apparent resilience under FGSM may stem from suboptimal single-step attack directions, as the stronger iterative PGD attack is able to overcome this advantage.

Finally, we note that in Dataset 3 in Appendix A.1 under PGD, the monotonicity assumption is violated: ASR decreases for certain increases in ε . This non-monotonic behavior may result from PGD converging to suboptimal local perturbations at particular budget levels, which is expected since we use a heuristic attack method.

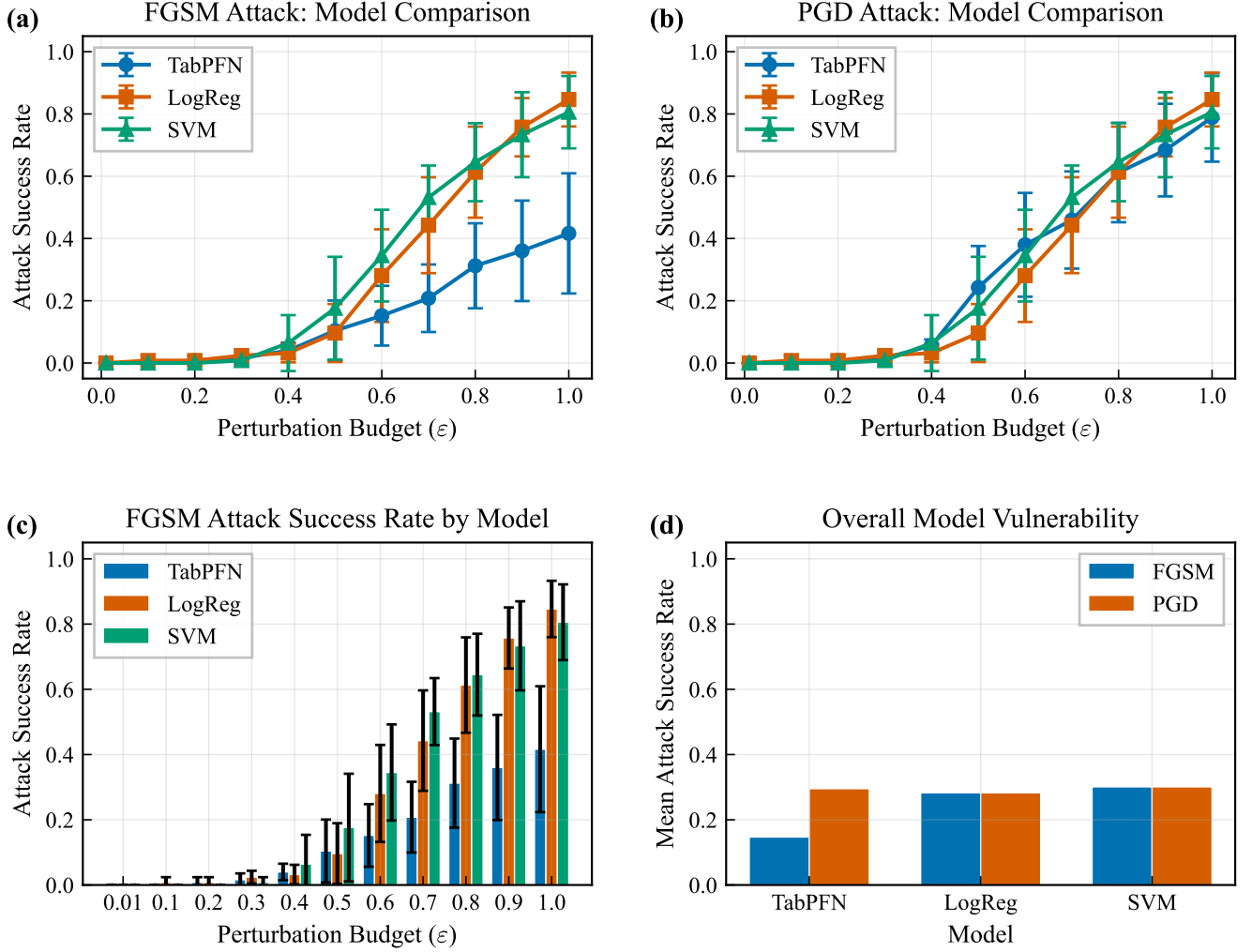


Figure 7: Comparative adversarial robustness of TabPFN, logistic regression, and a linear SVM on synthetic datasets. Panels (a) and (b) show the mean attack success rate as a function of the perturbation budget ϵ under FGSM and PGD, respectively, with error bars indicating one standard deviation across datasets. Panel (c) presents the FGSM results as a grouped bar chart for each ϵ value. Panel (d) summarises overall model vulnerability by averaging the attack success rate across all perturbation budgets for both attack types. At small perturbation budgets ($\epsilon \leq 0.5$), all three models exhibit comparably low attack success rates under both FGSM and PGD. For larger ϵ , TabPFN remains notably more robust than logistic regression and the linear SVM under FGSM, but this advantage disappears under the stronger iterative PGD attack, where all three models reach similar ASR levels.

Comparison With Neural Networks Next, we compare TabPFN against both a standard MLP and an adversarially trained MLP on three synthetic datasets. The MLP architecture consists of three linear layers with hidden dimensions 64 and 32, interleaved with ReLU activations, mapping 5 input features to 2 output classes. Both MLPs are trained for 100 epochs using the Adam optimizer with a learning rate of 10^{-3} and cross-entropy loss. The adversarially trained variant follows the approach of Madry et al. [MMS⁺17]. At each epoch, training samples are replaced by adversarial examples generated via 10-step PGD with perturbation budget $\varepsilon = 0.1$, step size $\alpha = 0.01$, and the ℓ_∞ norm. TabPFN is used in its pretrained form without any fine-tuning. PGD attacks use 10 steps, 5 random restarts with uniform initialization within the ℓ_∞ -ball, and a step size of $\alpha = 2\varepsilon/10$. This experiment investigates whether the robustness of TabPFN, which is not explicitly trained to resist adversarial perturbations, is comparable to that of a model specifically hardened through adversarial training. We report mean attack success rates with error bars, with per-dataset results deferred to Appendix A.2. The results are shown in Figure 8.

Under FGSM, TabPFN again exhibits the lowest ASR for larger perturbation budgets, while the standard MLP and adversarially trained MLP follow similar trajectories. Notably, adversarial training provides only a marginal benefit, with the adversarially trained MLP performing comparably to its standard counterpart, suggesting that the adversarial training regime may not have been sufficiently aggressive to confer meaningful robustness on these synthetic datasets. Under PGD, all three models become more vulnerable at large ε , but TabPFN displays a clear disadvantage up to approximately $\varepsilon = 0.7$. Beyond this point, the gap narrows, and the error bars overlap substantially. Panel (d) summarises the overall picture: TabPFN exhibits the lowest mean ASR under FGSM, while the opposite is true under PGD.

Robustness Distribution of TabPFN To obtain a more fine-grained picture of TabPFN’s adversarial vulnerability, we examine its robustness distribution as defined in Section 3.1.2. Specifically, we investigate how the distribution changes with feature dimension, considering datasets with 5, 15, and 25 features under both FGSM and PGD attacks, where we scope the binary search in $[0, 4]$. On the described hardware configuration, the full experiment completes in approximately 19 hours and 6 minutes.

The results are shown in Figure 9. We see that the distributions are tightly concentrated across datasets for all feature dimensions for PGD, with the curve shifting leftward as the number of features increases. This indicates that TabPFN becomes more vulnerable to iterative attacks in higher dimensions, consistent with the larger perturbation space available to the attacker. Under FGSM, the same leftward shift is visible, but the distributions exhibit substantially greater variance between datasets, and a notable fraction of samples remains robust even at large ε . This again suggests that FGSM fails to find optimal attack directions for certain samples, particularly in lower dimensions.

6.2 Real World Datasets

To see whether the results found on the artificial dataset hold for real-world data, we repeat the comparison experiment on the Wisconsin Breast Cancer dataset [WMSS93]. This dataset comprises

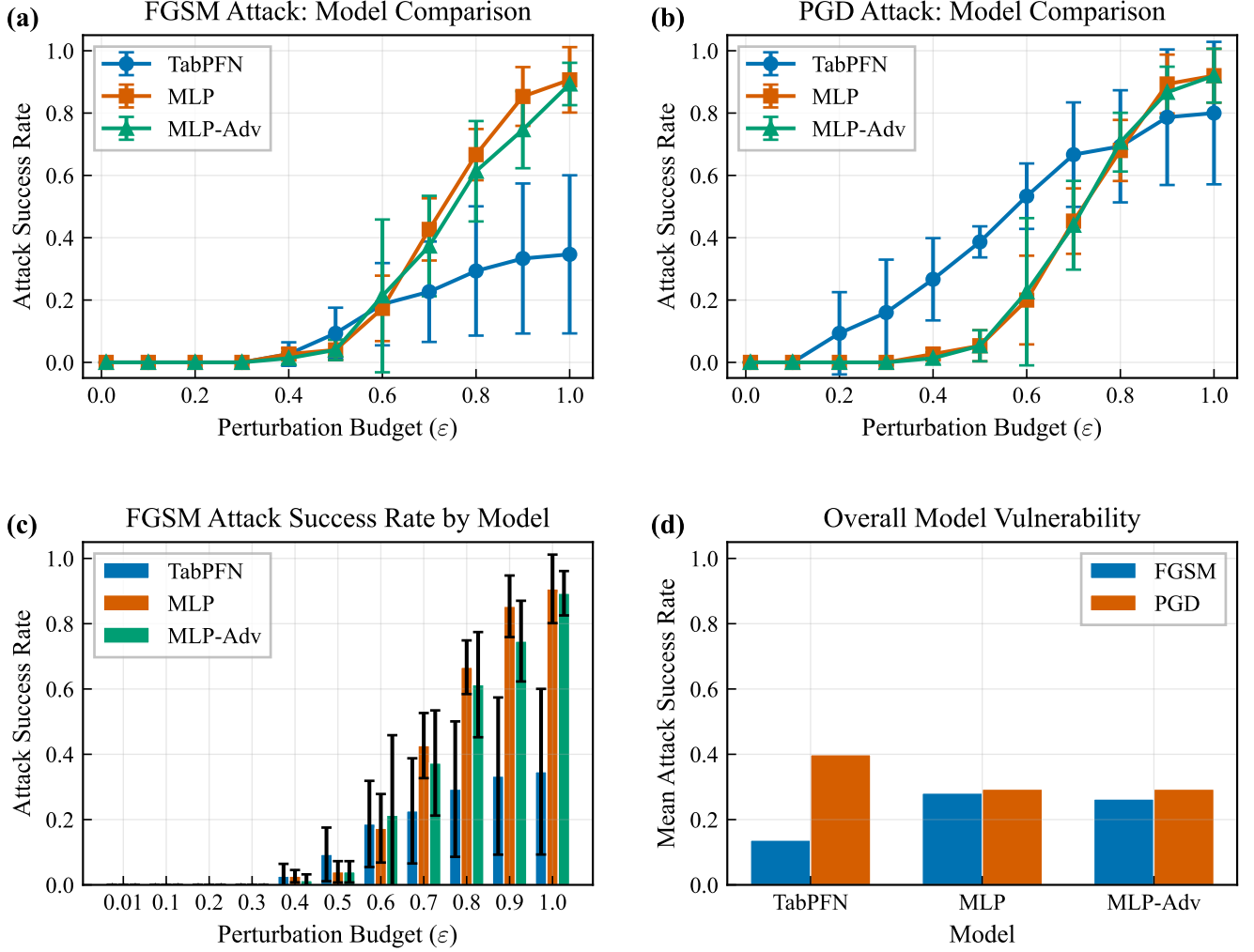


Figure 8: Comparative adversarial robustness of TabPFN, a standard MLP, and an adversarially trained MLP (MLP-Adv) on synthetic datasets. Panels (a) and (b) show the mean attack success rate as a function of the perturbation budget ϵ under FGSM and PGD, respectively, with error bars indicating one standard deviation across datasets. Panel (c) presents the FGSM results as a grouped bar chart for each ϵ value. Panel (d) summarises overall model vulnerability by averaging the attack success rate across all perturbation budgets for both attack types.

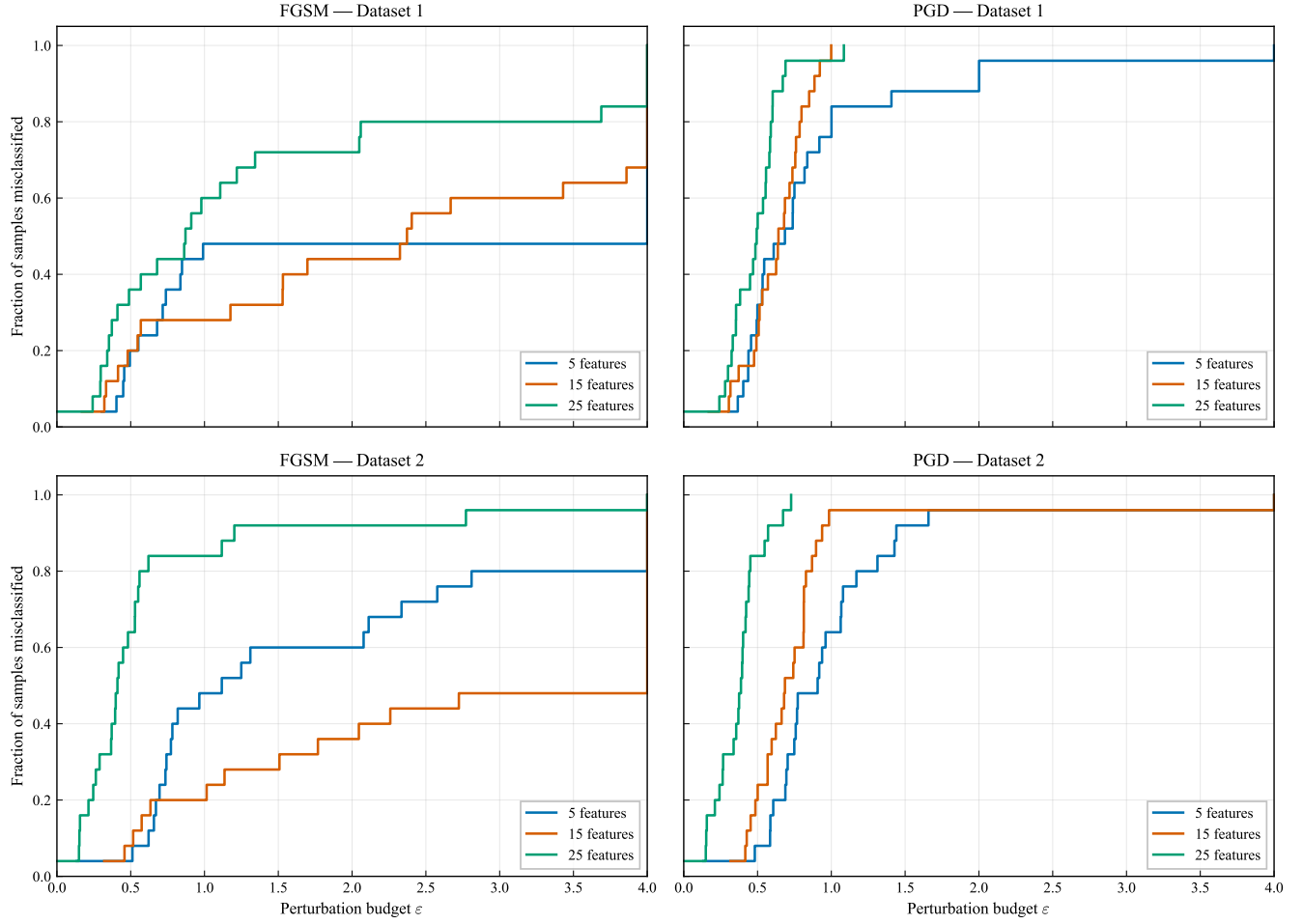


Figure 9: Robustness distribution of TabPFN under FGSM (left) and PGD (right) for datasets with 5, 15, and 25 features (top to bottom). Each curve shows the fraction of test samples misclassified as a function of the perturbation budget ϵ , with individual datasets shown alongside the mean and one standard deviation band. A curve that rises more steeply indicates greater vulnerability.

569 instances with 30 continuous features and binary class labels indicating benign or malignant tumors. Classifiers typically achieve near-perfect performance [Aga18] on this dataset, and are therefore useful for evaluating adversarial robustness in real-world tabular classification settings where small perturbations might lead to clinically significant misclassifications.

Breast Cancer Dataset Sixth, we extend the comparative robustness evaluation of TabPFN against classical methods to a real-world setting using the Wisconsin Breast Cancer dataset. By repeating the same experimental protocol on clinical data, we assess whether the robustness patterns observed on synthetic datasets generalize to a naturally occurring feature space with correlated and heterogeneously scaled attributes. This experiment takes approximately 16 hours and 12 minutes to complete on the specified hardware. Additionally, we inspect the empirical robustness distribution of TabPFN to see whether it corresponds to the other results. This completes in 16 hours and 17 minutes.

The results are shown in Figure 10. We observe that on the Wisconsin Breast Cancer dataset, TabPFN demonstrates stronger robustness than both logistic regression and the linear SVM. Under FGSM, TabPFN’s ASR remains below 0.1 across the entire range of perturbation budgets, while the classical models reach ASR values above 0.9 at $\varepsilon = 1.0$. Under PGD, TabPFN’s advantage is partially reduced but still substantial, with its ASR reaching approximately 0.3 at $\varepsilon = 1.0$ compared to near-complete vulnerability for logistic regression and the SVM. This robustness gap is further evident in panel (d): despite all three models achieving comparable clean accuracy, TabPFN retains an adversarial accuracy of approximately 0.68 under PGD at $\varepsilon = 1.0$, while the classical models degrade to a near-chance level. Unlike the synthetic experiments, where PGD largely eliminated TabPFN’s advantage over classical methods, the robustness gap persists on this real-world dataset under both attack types. It is not immediately clear why this advantage persists, particularly given the high feature dimensionality of the Breast Cancer dataset and our earlier finding that vulnerability increases with the number of features. We hypothesize that the structure of the real-world data plays a decisive role. Unlike synthetic datasets, the Breast Cancer features are likely to contain redundancies and correlations that TabPFN can exploit to form more robust decision boundaries. Investigating this hypothesis further is left to future work.

To conclude our analysis, we report the findings of our final experiment, shown in Figure 11. The robustness distribution of TabPFN on the Breast Cancer dataset confirms the patterns observed in the ASR analysis. Under PGD, the distribution rises steadily and reaches about 80% misclassification at approximately $\varepsilon = 4$, indicating that while TabPFN is substantially more robust than the classical models, no sample remains robust to arbitrarily large perturbations. Under FGSM, a large fraction of samples remains correctly classified even at $\varepsilon = 4.0$, further supporting the earlier observation that single-step attacks fail to find effective perturbation directions for TabPFN on this dataset.

Compare Example Transferability Question: Do we want to only compare whether models fail on the same instances, or also if an adversarial example of one also induces missclassification in the other model?

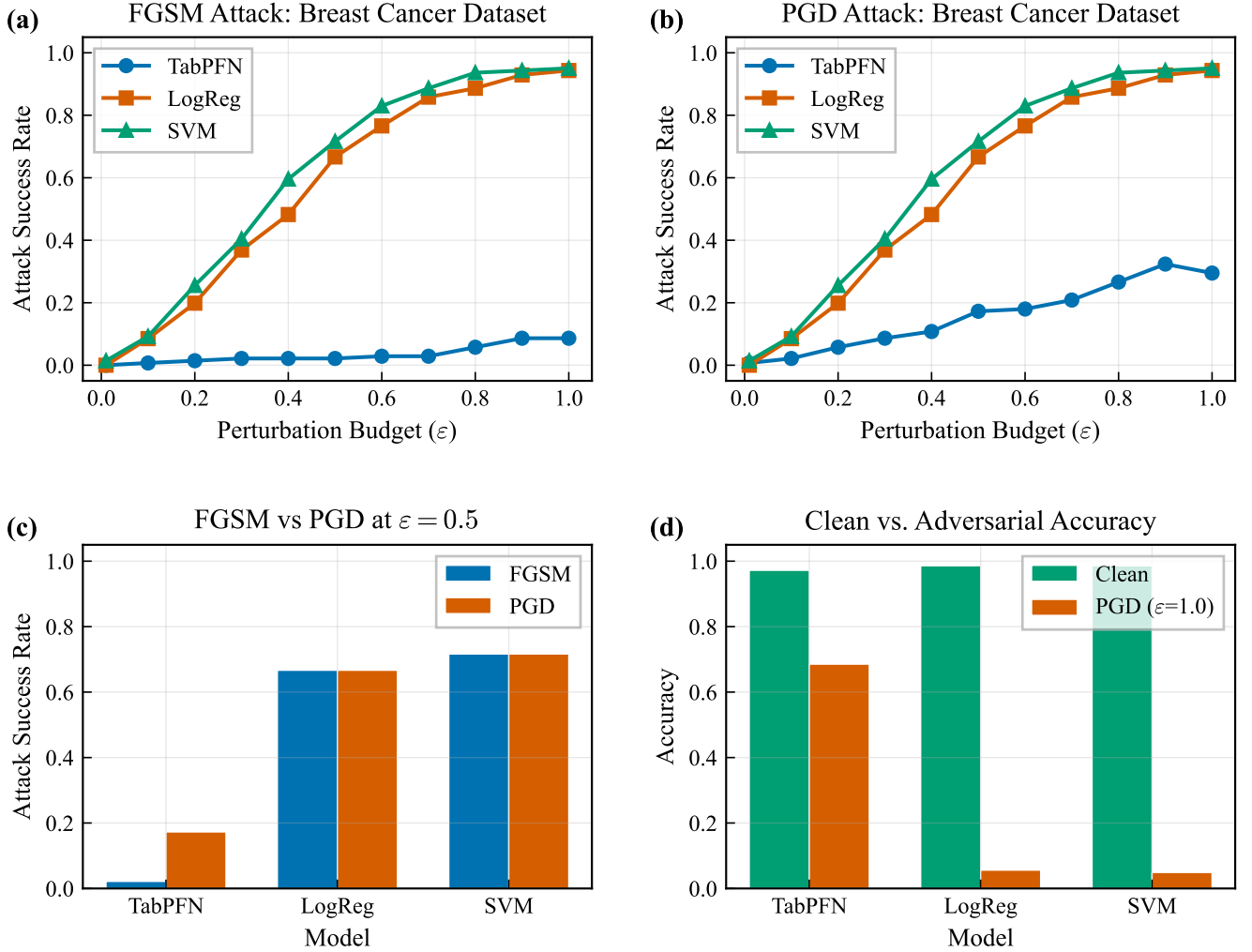


Figure 10: Adversarial robustness evaluation on the Wisconsin Breast Cancer dataset for TabPFN, logistic regression, and a linear SVM. (a) Attack success rate (ASR) as a function of perturbation budget ϵ under FGSM, showing that TabPFN remains largely unaffected across the full range of ϵ , while logistic regression and the SVM exhibit steeply increasing vulnerability. (b) ASR under PGD (5 steps, 5 restarts), where TabPFN’s robustness advantage persists but is partially reduced, reaching approximately 0.3 ASR at $\epsilon = 1.0$. (c) Direct comparison of FGSM and PGD attack success at $\epsilon = 0.5$. (d) Clean versus adversarial accuracy under PGD at $\epsilon = 1.0$. Despite comparable clean accuracy across all models, TabPFN retains approximately 0.68 adversarial accuracy, while logistic regression and the SVM degrade to near chance level.

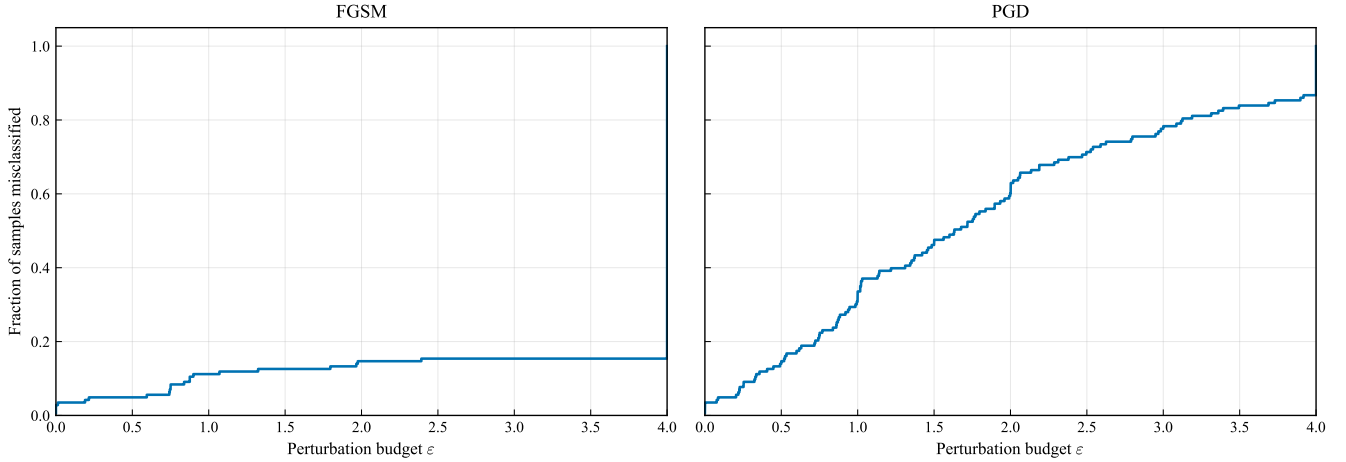


Figure 11: Robustness distribution of TabPFN on the Wisconsin Breast Cancer dataset under FGSM (left) and PGD (right). Each curve shows the fraction of correctly classified test samples for which an adversarial example is found as a function of the perturbation budget ε .

7 Conclusions and Further Research

This thesis investigated the local adversarial robustness of TabPFN on numerical binary classification tasks. Using FGSM and PGD attacks, we compared TabPFN against logistic regression, a linear SVM, and both standard and adversarially trained MLPs on synthetic datasets, and against the classical models on the Wisconsin Breast Cancer dataset. We additionally computed empirical robustness distributions to characterize TabPFN’s vulnerability at a finer granularity than aggregate attack success rates alone.

Our experiments revealed three main findings. First, on synthetic data, TabPFN consistently exhibited lower attack success rates than all baselines under FGSM, but this advantage largely disappeared under the stronger iterative PGD attack, suggesting that TabPFN’s gradient landscape may obstruct single-step attacks without conferring true robustness. Second, robustness distributions showed that vulnerability increases with feature dimensionality, consistent with the larger perturbation space available to the attacker in higher dimensions. Third, on the Breast Cancer dataset, TabPFN retained a substantial robustness advantage over classical methods under both attack types, a finding we hypothesize is related to the redundancy and correlation structure present in real-world features.

Several limitations should be noted. Our analysis relies on heuristic attacks, which provide upper bounds on robustness rather than formal guarantees. The adversarially trained MLP baseline used a relatively mild training regime, which may explain its limited improvement over the standard MLP. Furthermore, our real-world evaluation is restricted to a single dataset, limiting the generalizability of the findings. Finally, we assumed strictly numerical data and no constraints on the input, whereas in practice, data can be of mixed types and valid inputs may be constrained to a bounded domain, which could affect both the attack strategies and the resulting robustness estimates.

Several directions for future research present themselves. First, extending the evaluation to a broader

collection of real-world tabular datasets would strengthen the generalizability of our conclusions. Second, once formal verification methods are available and feasible for transformer architectures, incorporating them would complement the heuristic analysis with provable robustness bounds. It would also be interesting to see how AICL, an adversarial training mechanism introduced in [DST+25], would affect the results. Finally, relaxing the constraints on the data would bring the robustness analysis closer to practical deployment settings, where more sophisticated attack methods are required.

References

- [Aga18] Abien Fred M Agarap. On breast cancer detection: an application of machine learning algorithms on the wisconsin diagnostic dataset. In *Proceedings of the 2nd international conference on machine learning and soft computing*, pages 5–9, 2018.
- [BBHVR25] Annelot Bosman, Aaron Berger, Holger H Hoos, and Jan N Van Rijn. Robustness distributions in neural network verification. *Journal of Artificial Intelligence Research*, 83, 2025.
- [BDBV21] Gregory Bonaert, Dimitar I Dimitrov, Maximilian Baader, and Martin Vechev. Fast and precise certification of transformers. In *Proceedings of the 42nd ACM SIGPLAN international conference on programming language design and implementation*, pages 466–481, 2021.
- [CJB⁺24] Marcelo Contreras, Aayush Jain, Neel P Bhatt, Arunava Banerjee, and Ehsan Hashemi. A survey on 3d object detection in real time for autonomous driving. *Frontiers in Robotics and AI*, 11:1212070, 2024.
- [DST⁺25] Mohamed Djilani, Thibault Simonetto, Karim Tit, Florian Tambon, Paul Récamier, Salah Ghamizi, Maxime Cordy, and Mike Papadakis. On the robustness of tabular foundation models: Test-time attacks and in-context defenses. *arXiv preprint arXiv:2506.02978*, 2025.
- [GSS14] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- [HMEH23] Noah Hollmann, Samuel Müller, Katharina Eggersperger, and Frank Hutter. Tabpfn: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations 2023*, 2023.
- [KBD⁺17] Guy Katz, Clark Barrett, David L Dill, Kyle Julian, and Mykel J Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks. In *International conference on computer aided verification*, pages 97–117. Springer, 2017.
- [LFK⁺19] Xiaoxuan Liu, Livia Faes, Aditya U Kale, Siegfried K Wagner, Dun Jack Fu, Alice Bruynseels, Thushika Mahendiran, Gabriella Moraes, Mohith Shamdas, Christoph Kern, et al. A comparison of deep learning performance against health-care professionals in detecting diseases from medical imaging: a systematic review and meta-analysis. *The lancet digital health*, 1(6):e271–e297, 2019.
- [MMS⁺17] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- [OC21] Alice J O’Toole and Carlos D Castillo. Face recognition by humans and machines: three fundamental advances from deep learning. *Annual Review of Vision Science*, 7(1):543–570, 2021.

- [SZS⁺13] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.
- [TXT17] Vincent Tjeng, Kai Xiao, and Russ Tedrake. Evaluating robustness of neural networks with mixed integer programming. *arXiv preprint arXiv:1711.07356*, 2017.
- [WMSS93] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository, 1993. DOI: <https://doi.org/10.24432/C5DW2B>.
- [YLC] HJ Ye, SY Liu, and WL Chao. A closer look at tabPFN v2: Strength, limitation, and extension. arXiv 2025. *arXiv preprint arXiv:2502.17361*.

A Detailed Results for Synthetic Robustness Experiments

This appendix presents the per-dataset results underlying the aggregated figures in Section ?? . Each figure shows the attack success rate as a function of the perturbation budget ε for every individual synthetic dataset, complementing the mean and standard deviation reported in the main text.

A.1 Classical Methods Comparison

Figure 12 shows the per-dataset breakdown of the classical methods comparison. The aggregate trends reported in Section ?? are consistent across most datasets: TabPFN exhibits lower ASR than logistic regression and the linear SVM under FGSM, while all three models converge under PGD. Individual datasets occasionally deviate from this pattern, most notably Dataset 3, where a non-monotonic ASR under PGD is visible.

A.2 Neural Network Comparison

Figure 13 presents the per-dataset results for the MLP comparison. The individual datasets confirm the aggregate finding that TabPFN is less vulnerable to FGSM attacks, while it consistently performs worse under PGD. The adversarially trained MLP does not show a clear advantage over the standard MLP on any individual dataset.

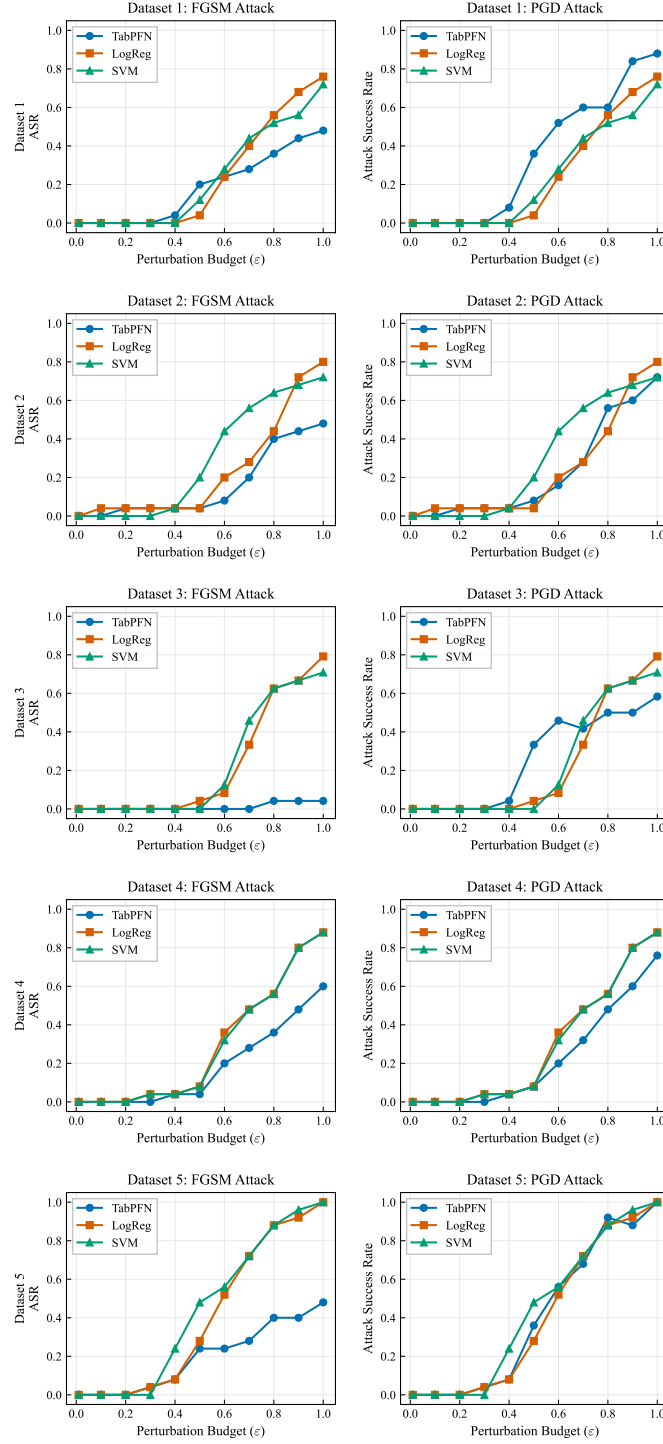


Figure 12: Attack success rate (ASR) as a function of perturbation budget (ϵ) for FGSM (left column) and PGD (right column, 5 steps, 5 restarts) attacks on TabPFN, logistic regression, and a linear SVM, evaluated across five synthetic datasets. TabPFN appears more robust for larger ϵ under FGSM, but all three models appear similar for all ϵ under PGD.

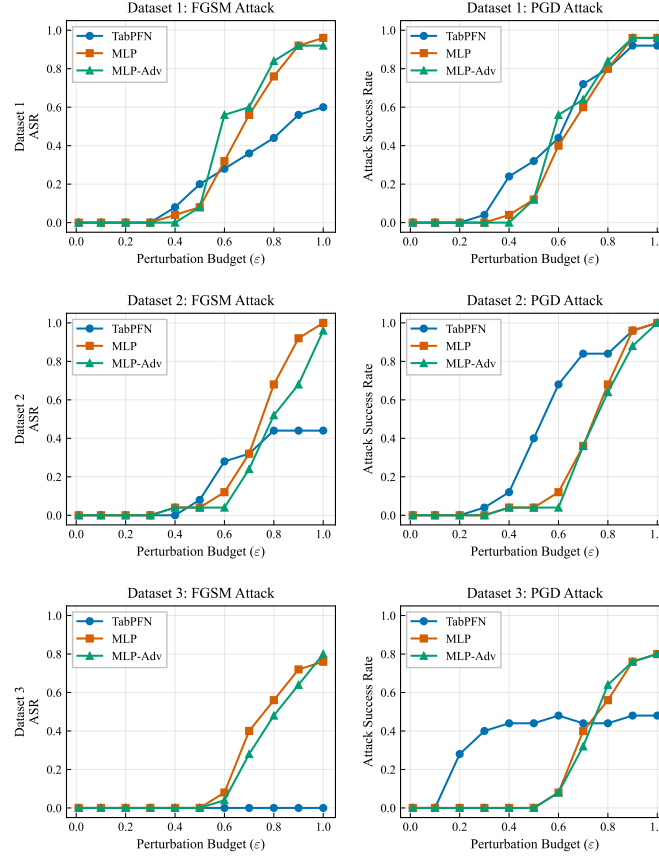


Figure 13: Attack success rate (ASR) as a function of perturbation budget (ϵ) for FGSM (left column) and PGD (right column, 5 steps, 5 restarts) attacks on TabPFN, MLP, and adversarially trained MLP evaluated across three synthetic datasets. TabPFN appears more robust for larger ϵ under FGSM, but less robust for smaller ϵ under PGD. There is no clear difference between the adversarially trained MLP and the normal MLP, possibly indicating that the training set is not large enough.